

PAPER → CODE · ARXIV:2203.15556

# Chinchilla

## Training Compute-Optimal LLMs

---

Every equation of Hoffmann et al. (DeepMind, 2022), extracted and explained parameter by parameter — the parametric loss law, the IsoFLOP experiment, the closed-form frontier, and the correction of Kaplan's scaling laws that shrank every model after it. Loss  $L$  in nats/token on MassiveText;  $C = 6 \cdot N \cdot D$  FLOPs.

**The math explained** – companion to the interactive site

Original authors: J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, et al.

Visualised & annotated by **Abdulsamad Teniola Muyideen**

Core equations 2-4 & 10-11 · Tables 2-3 · 4 computed figures · runnable at  
Nanogpt/nano\_chinchilla.py

# The parametric loss law

L(N,D) – THREE TERMS EXPLAIN EVERYTHING

(2 / 10)

$$\hat{L}(N,D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} = 1.69 + \frac{406.4}{N^{0.34}} + \frac{410.7}{D^{0.28}}$$

**What it does.** Predicts the final training loss of a Transformer with N parameters trained on D tokens (single epoch). A classical risk decomposition: E is the Bayes risk — the entropy of natural text, which no model of any size beats; A/N^α is the capacity error of a finite network; B/D^β is the estimation error of finite data seen once. Fit to over 400 DeepMind training runs.

| SYMBOL | MEANING                                  | VALUE       |
|--------|------------------------------------------|-------------|
| E      | irreducible loss floor (entropy of text) | 1.69 nats   |
| A, α   | capacity term scale and exponent         | 406.4, 0.34 |
| B, β   | data term scale and exponent             | 410.7, 0.28 |
| N, D   | parameters; training tokens              | —           |

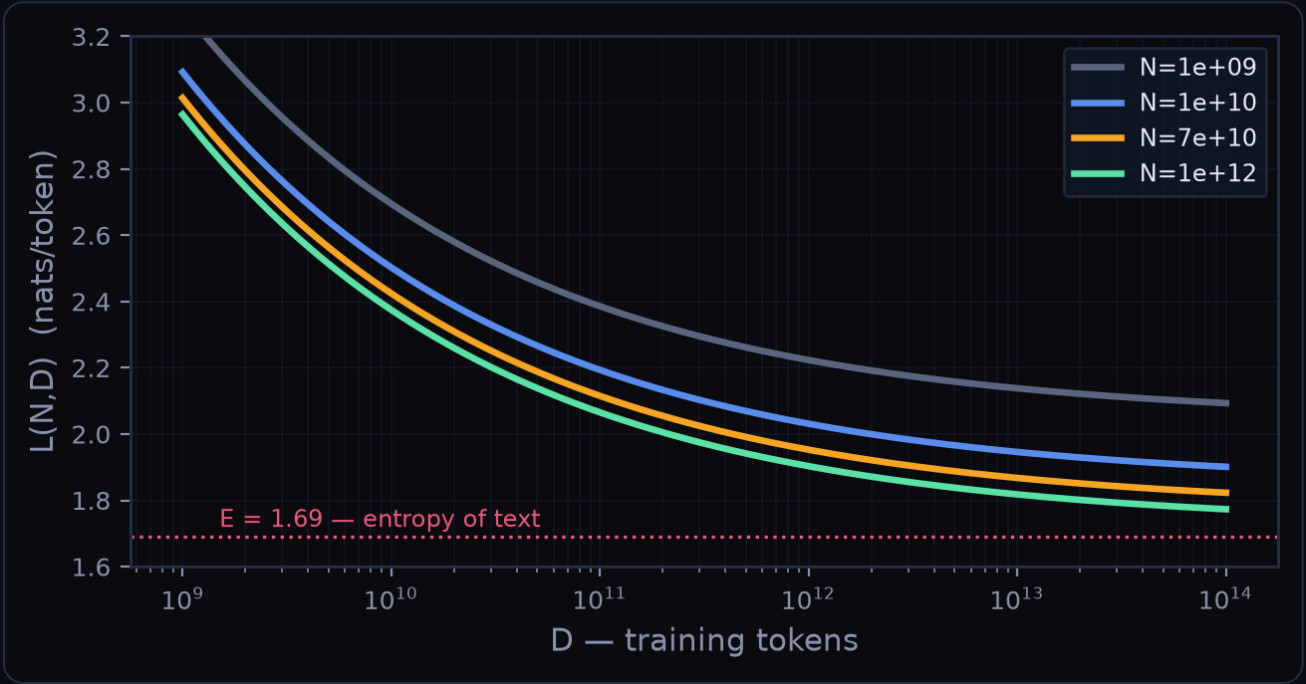
THE FITTING OBJECTIVE

(3 / 11)

$$\min_{a,b,e,\alpha,\beta} \sum_{\text{runs } i} \text{Huber}_\delta \left( \text{LSE}(a - \alpha \log N_i, b - \beta \log D_i, e) - \log L_i \right)$$

**What it does.** How (A,B,E,α,β) are estimated: minimise a Huber loss (δ=10<sup>-3</sup>) between predicted and observed log-loss with L-BFGS, from a grid of initialisations. Huber = robust to outliers, so the noisy low-compute runs don't drag the fit; log-space keeps A=e^a, B=e^b, E=e^e positive. Runnable at tiny scale in Nanogpt/nano\_chinchilla.py.

| SYMBOL | MEANING              | VALUE            |
|--------|----------------------|------------------|
| LSE    | log-sum-exp operator | —                |
| δ      | Huber threshold      | 10 <sup>-3</sup> |



Loss vs tokens for several model sizes – every curve floors at its capacity limit; nothing beats E=1.69.

# IsoFLOP profiles

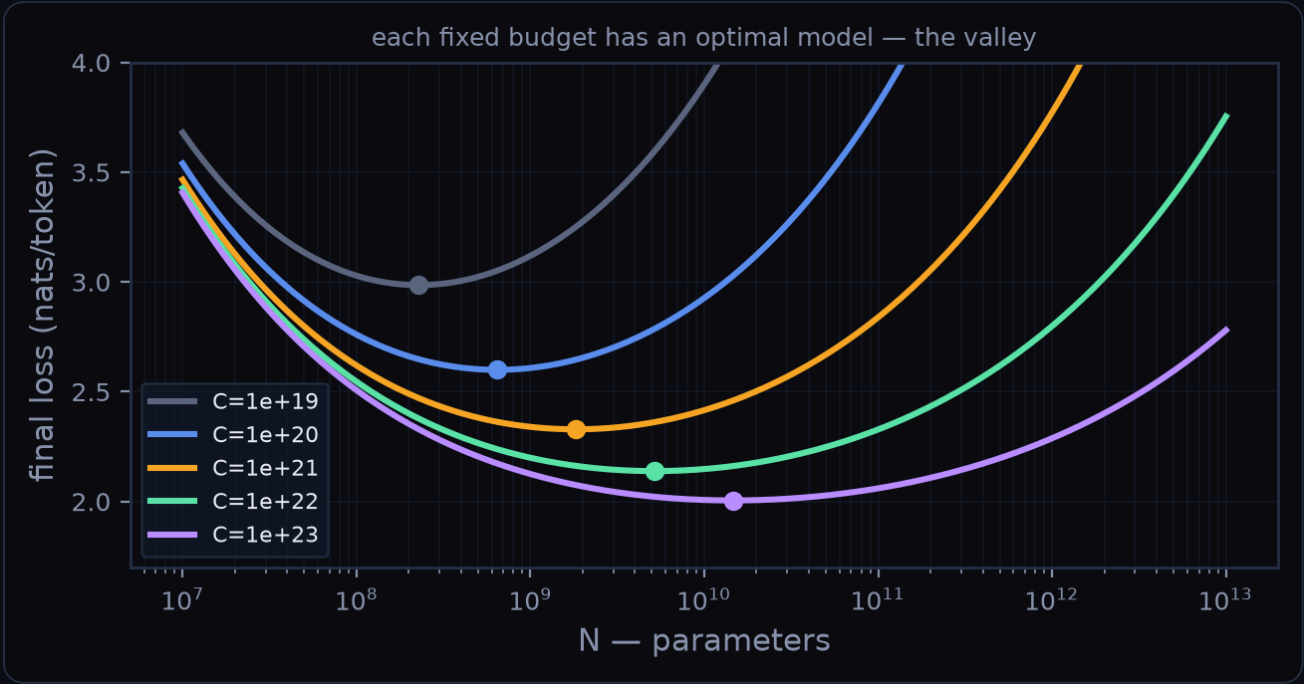
THE VALLEY EXPERIMENT

(§3.2)

$$\text{IsoL}(N; C) = \hat{L}\left(N, \frac{C}{6N}\right), \quad C = 6ND \text{ fixed}$$

**What it does.** The question a lab actually faces: given C FLOPs, what model should I train? Fix the budget, sweep model size — every extra parameter costs tokens ( $D = C/6N$ ). Too small a model can't represent the distribution; too big a model is undertrained. The paper ran 9 budgets ( $6 \times 10^{18} \rightarrow 3 \times 10^{21}$  FLOPs), fit a parabola to each valley in log N, then a power law through the minima. The crucial methodological fix over Kaplan 2020: the cosine learning-rate schedule is stretched to match each run's token horizon — reusing one schedule for all horizons (as Kaplan did) under-credits the long-data runs and inflates the model-size exponent.

| SYMBOL       | MEANING                                       | VALUE |
|--------------|-----------------------------------------------|-------|
| $C$          | training FLOPs, $C \approx 6 \cdot N \cdot D$ | —     |
| $N_{opt}(C)$ | valley bottom — optimal model at budget C     | —     |



IsoFLOP valleys computed from the fitted law; dots mark  $N_{opt}(C)$  — the bottoms line up in log-log space.

# The compute-optimal frontier

OPTIMAL ALLOCATION

(4)

$$N_{opt}(C) = G \left(\frac{C}{6}\right)^a, \quad D_{opt}(C) = G^{-1} \left(\frac{C}{6}\right)^b, \quad G = \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}}, \quad a = \frac{\beta}{\alpha+\beta}, \quad b = \frac{\alpha}{\alpha+\beta}$$

**What it does.** Minimise L(N,D) subject to 6ND = C and the optimum falls out in closed form. With the fitted  $\alpha, \beta$ :  $a \approx 0.46$ ,  $b \approx 0.54$  — model size and data scale in near-equal proportion with compute. Approaches 1 and 2 give  $a = b \approx 0.50$  independently. All three contradict Kaplan's  $a = 0.73, b = 0.27$ .

| SYMBOL        | MEANING                           | VALUE              |
|---------------|-----------------------------------|--------------------|
| <i>a</i>      | N_opt exponent (three approaches) | 0.50 / 0.49 / 0.46 |
| <i>b</i>      | D_opt exponent                    | 0.50 / 0.51 / 0.54 |
| <i>Kaplan</i> | the claim being corrected         | a=0.73, b=0.27     |

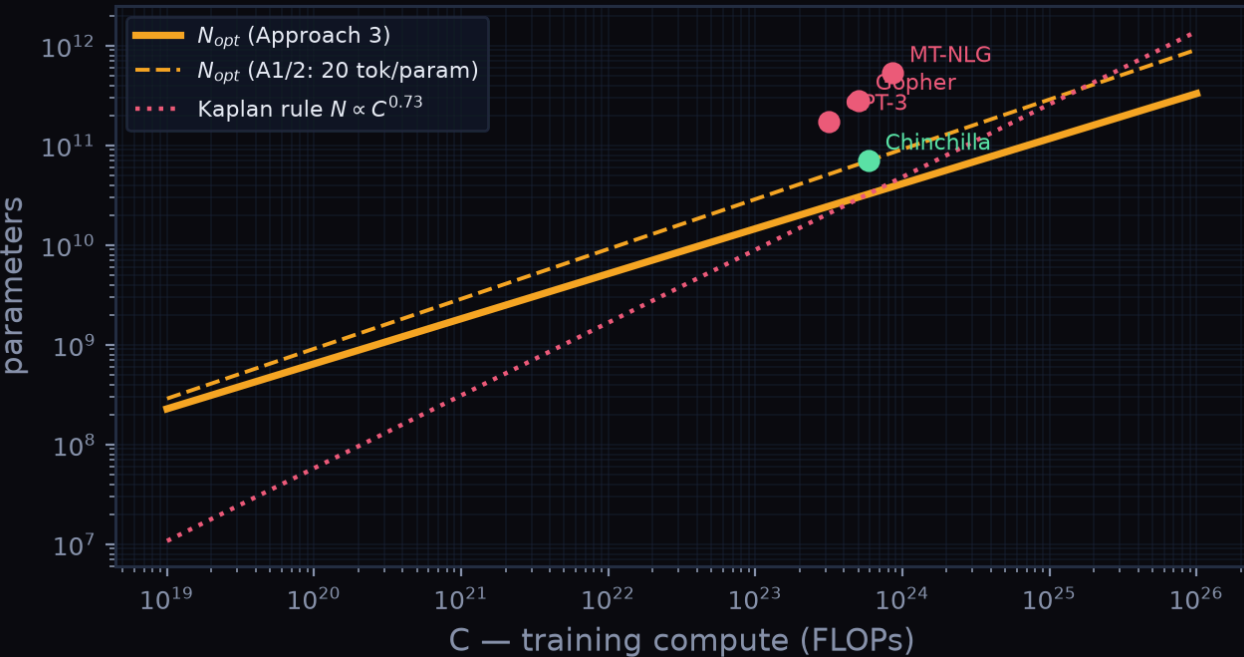
THE 20-TOKENS-PER-PARAMETER RULE

(Table 3)

$$D_{opt} \approx 20 \times N_{opt}$$

**What it does.** With  $a = b = 0.5$  (Approaches 1–2) the ratio D/N is a constant, and Table 3 pins it near 20 across every practical budget: 400M→8B tokens, 1B→20.2B, 10B→205B, 67B→1.5T, 175B→3.7T. Chinchilla itself: 1.4T/70B = 20 exactly. Subtlety: the Approach-3 closed form has  $a < b$ , so its ratio drifts upward (~40–60 in range) — which is why Approach 3 predicts the smallest optimal models of the three.

| SYMBOL     | MEANING                        | VALUE                  |
|------------|--------------------------------|------------------------|
| <i>D/N</i> | frontier ratio, Approaches 1–2 | ≈ 20                   |
| <i>D/N</i> | frontier ratio, Approach 3     | ~40–60, slowly growing |



The frontier vs the Kaplan rule, with real models plotted: Chinchilla on the line; GPT-3, Gopher, MT-NLG far above it.

# The verdict

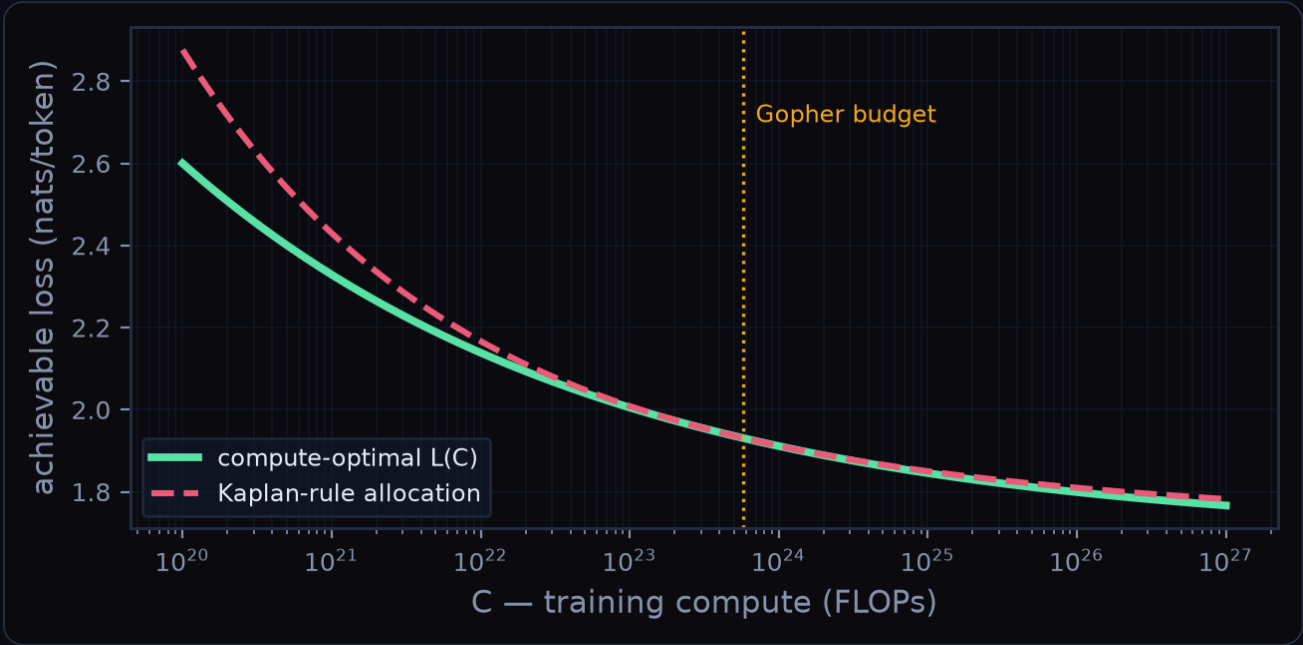
PROOF BY TRAINING

(§4)

Chinchilla:  $70\text{B} \times 1.4\text{T}$  tokens vs Gopher:  $280\text{B} \times 300\text{B}$  tokens (same  $C \approx 5.8 \times 10^{23}$ )

**What it does.** The paper's hypothesis test: spend Gopher's exact budget the compute-optimal way — 4× smaller, 4× more data. Chinchilla outperforms Gopher (and GPT-3, Jurassic-1, MT-NLG) essentially everywhere: +7.6% average on MMLU (67.5%), better bits-per-byte on all 19 Pile subsets tested, better on the vast majority of BIG-bench tasks — while being 4× cheaper at inference and fine-tuning forever after. Given  $\sim 10^{25}$  FLOPs, the optimal Gopher-class model would be  $\sim 280\text{B}$  on 6.8T tokens; a 1T-parameter model is not optimal below  $\sim 10^{26}$  FLOPs.

| SYMBOL           | MEANING                          | VALUE      |
|------------------|----------------------------------|------------|
| <i>MMLU</i>      | Chinchilla 67.5% vs Gopher 60.0% | +7.5 pts   |
| <i>inference</i> | cost ratio vs Gopher             | 4× cheaper |



Feeding both allocation rules into the same loss surface: the Kaplan-rule penalty compounds with compute.